UNIVERSIDAD NACIONAL DE LOJA

Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables

Carrera de Computación — Ciclo 6

ASIGNATURA:
Teoría de Autómatas y Computabilidad Avanzada (E2C6A1)

EVALUACIÓN SUMATIVA 1 — UNIDAD 1

Informe Técnico: Monitor de Bitácoras de Eventos con Autómatas Finitos para Sistema IoT de Casilleros Inteligentes

Nombre del equipo: AutomataTech
Docente: Ing. Genoveva Suing
Fecha de entrega: 28 de octubre de 2026

Integrantes y Roles:

N.º	Estudiante	Rol Asignado	Usuarios Github
1	Israel Rodriguez	Responsable del modelo formal	IsraelSRodriguezT
2	Sebastian Sanchez	Desarrollador del monitor (Python)	Sebaslsst
3	Carlos Granda	Responsable de datos	Carlosjosu
4	Isaac Cartuche	Documentador y redacción técnica	IsaacCartuche
5	Tayron Morales	Responsable de pruebas de calidad	Tayron2024

Enlace a repositorio Github:
https://github.com/IsaacCartuche/Evaluacion_Sumativa_U1_Automatas.git

Loja – Ecuador
2026–2027

1. Introducción

En la actualidad, los sistemas de casilleros inteligentes son ampliamente utilizados en recintos universitarios, centros comerciales y estaciones de transporte para el depósito seguro de pertenencias. Estos sistemas operan mediante una arquitectura híbrida compuesta por una aplicación web interactiva (donde los usuarios reservan casilleros, obtienen credenciales dinámicas y gestionan sus sesiones) y dispositivos de hardware IoT.

Cada acción realizada en el sistema —desde la solicitud de reserva en la web hasta la lectura del sensor infrarojo de la puerta— se registra cronológicamente en bitácoras digitales. Una sesión de usuario se compone de una secuencia ordenada de dichos eventos. Este proyecto resuelve el problema de la vigilancia y detección automática de secuencias anómalas (fuerza bruta en códigos de acceso, aperturas forzadas y pertenencias olvidadas al finalizar reservas) mediante el modelado formal con autómatas finitos deterministas (AFD) y no deterministas (AFND), implementando un monitor de eventos eficiente en Python.

2. Modelo formal

2.1. Diccionario de eventos (Alfabeto Σ)

Se define el alfabeto del sistema Σ compuesto por 8 símbolos discretos, donde cada símbolo representa una acción registrada por la plataforma web o los sensores IoT.

Tabla 1: Diccionario de eventos del sistema de casilleros inteligentes.

Símbolo	Evento	Descripción	Relevancia
R	RESERVA	Usuario realiza la reserva desde la aplicación web.	Relevante
C	COD_CORRECTO	Se ingresa el código PIN único correcto en el casillero.	Relevante
I	COD_INCORRECTO	Se ingresa un código PIN erróneo en el teclado digital.	Relevante
A	PUERTA_ABIERTA	El sensor detecta la puerta abierta.	Relevante
E	PUERTA_CERRADA	El sensor detecta la puerta cerrada.	Relevante
O	OCUPADO	Sensor óptico detecta objeto dentro del casillero.	Relevante
F	FIN_RESERVA	Finalización de la reserva (por app o caducidad).	Relevante
L	CONSULTA_APP	Usuario consulta el estado en la app (sin impacto).	Irrelevante (Bucle)

2.2. Ejemplo de sesión convertida en cadena

Dada una sesión de uso normal S-023, donde un usuario reserva el casillero desde la app, consulta el mapa, ingresa el código correcto, abre la puerta, guarda su mochila (el sensor detecta presencia), cierra la puerta y posteriormente retira su pertenencia finalizando la reserva:

$$S\text{-}023 \longrightarrow w = R \cdot L \cdot C \cdot A \cdot O \cdot E \cdot C \cdot A \cdot E \cdot F \in \Sigma^*$$

2.3. Patrones a detectar

P1: **Intento de fuerza bruta en clave de acceso (Anomalía):** Tres intentos consecutivos de código incorrecto dentro de una misma sesión.

$$L_1 = \{w \in \Sigma^* \mid w \text{ contiene la subsecuencia } III\}$$

Cadena válida ($w \in L_1$): $R \cdot I \cdot I \cdot I \cdot C$ — *Cadena no válida* ($w \notin L_1$): $R \cdot I \cdot I \cdot C \cdot I$

P2: **Apertura forzada sin autorización (Anomalía):** Apertura de la puerta (A) sin que se haya ingresado previamente un código correcto (C) en la sesión.

$$L_2 = \{w \in \Sigma^* \mid w \text{ contiene } A \text{ y no existe } C \text{ antes de la primera } A\}$$

Cadena válida ($w \in L_2$): $R \cdot I \cdot A$ — *Cadena no válida* ($w \notin L_2$): $R \cdot C \cdot A$

P3: **Pertenencia olvidada tras finalizar reserva (Anomalía):** El casillero registra estado ocupado (O) inmediatamente antes de la finalización de la reserva (F).

$$L_3 = \{w \in \Sigma^* \mid w \text{ contiene la subsecuencia } OF\}$$

Cadena válida ($w \in L_3$): $R \cdot C \cdot A \cdot O \cdot E \cdot F$ — *Cadena no válida* ($w \notin L_3$): $R \cdot C \cdot A \cdot O \cdot E \cdot C \cdot A \cdot E \cdot F$

3. Conjunto de datos

Para evaluar el monitor de eventos se optó por construir un generador de datos sintéticos en Python en lugar de recolectar bitácoras reales. Esta decisión se fundamenta en tres razones principales:

- **Privacidad y disponibilidad de información**: En entornos reales de casilleros inteligentes, las bitácoras registran datos confidenciales de los usuarios, como códigos PIN, horarios de entrega o patrones de uso cotidianos. Obtener estos registros exige procesos complejos de anonimización y autorizaciones institucionales. Además, los sistemas en producción suelen presentar una frecuencia muy baja de fallas o ataques reales, lo que dificulta obtener muestras representativas de comportamiento anómalo.
- **Control de proporciones y balance del problema**: Para validar que un autómata finito detecte correctamente cada patrón, es indispensable contar con un número suficiente de sesiones anómalas bien categorizadas. El generador permite fijar y controlar la proporción exacta de anomalías (entre el 15 % y el 20 %), garantizando que cada patrón ($P1$, $P2$ y $P3$) y caso de prueba se encuentre adecuadamente representado.
- **Reproducibilidad absoluta de los experimentos**: Al programar el generador con una semilla aleatoria fija (`seed=42`), cualquier persona o docente puede ejecutar el script `generador.py` y obtener exactamente el mismo conjunto de datos, con las mismas marcas de tiempo, orden de eventos y marcas de prueba. Esto asegura que los resultados reportados sean totalmente verificables y comparables.

3.1. Especificaciones de la muestra

- **Total de sesiones:** 400 sesiones (320 normales y 80 anómalas, correspondiente al 20 % de tasa de anomalías).
- **Longitud de sesiones:** De 4 a 18 eventos por sesión.
- **Inclusión de casos especiales:** Se incorporaron sesiones çasi anómalas"(como secuencias con 2 códigos incorrectos seguidos: $R \cdot I \cdot I \cdot C$) y múltiples eventos irrelevantes (L) intercalados.

3.2. Estructura del archivo CSV

Tabla 2: Fragmento del archivo de validación (`validacion.csv`).

id	fecha_hora	id_sesion	actor	evento	simbolo	etiqueta	tipo_patron
101	2026-10-25 08:00:12	S-001	U204	RESERVA	R	normal	N/A
102	2026-10-25 08:01:05	S-001	U204	COD_INCORRECTO	I	normal	N/A
103	2026-10-25 08:01:15	S-001	U204	COD_INCORRECTO	I	normal	N/A
104	2026-10-25 08:01:28	S-001	U204	COD_INCORRECTO	I	anómala	P1
105	2026-10-25 08:05:00	S-002	U112	RESERVA	R	normal	N/A
106	2026-10-25 08:05:30	S-002	U112	PUERTA_ABIERTA	A	anómala	P2

4. Diseño de los AFND

Nota: Los diagramas vectoriales elaborados en JFLAP se adjuntan en la carpeta `automatas/` *del repositorio. A continuación se presentan las definiciones formales y tablas de transición.*

4.1. Autómata AFND para Patrón P1 (Fuerza bruta: III)

Quíntupla: $M_1 = (Q_1, \Sigma, \delta_1, q_0, F_1)$ donde $Q_1 = \{q_0, q_1, q_2, q_3\}$ y $F_1 = \{q_3\}$.

- q_0: Estado inicial (esperando subsecuencia III).
- q_1: Se ha detectado 1 código incorrecto.
- q_2: Se han detectado 2 códigos incorrectos consecutivos.
- q_3: Estado final de aceptación (anomalía P1 detectada).

Tabla 3: Tabla de transiciones del AFND M_1.

Estado	Entrada I	Entrada $\Sigma \setminus \{I\}$
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$
$*q_3$	$\{q_3\}$	$\{q_3\}$

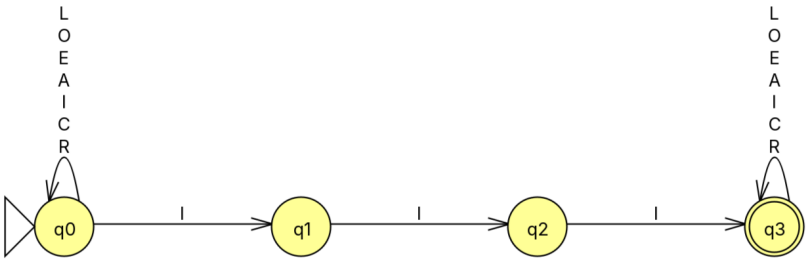


Figura 1: Autómata de fuerza bruta.

4.2. Autómata AFND para Patrón P2 (Apertura forzada)

Quíntupla: $M_2 = (Q_2, \Sigma, \delta_2, p_0, F_2)$ donde $Q_2 = \{p_0, p_1, p_2\}$ y $F_2 = \{p_2\}$.

- p_0: Estado inicial (esperando la secuencia que identifica una posible apertura forzada).
- p_1: Se ha detectado una parte de la secuencia correspondiente al patrón P2.
- p_2: Estado final de aceptación (se ha detectado la anomalía P2: apertura forzada).

Tabla 4: Tabla de transiciones del AFND M_2.

Estado	**Entrada** A	**Entrada** C	**Entrada** $\Sigma \setminus \{A, C\}$
$\rightarrow p_0$	$\{p_2\}$	$\{p_1\}$	$\{p_0\}$
p_1	$\{p_1\}$	$\{p_1\}$	$\{p_1\}$
$*p_2$	$\{p_2\}$	$\{p_2\}$	$\{p_2\}$

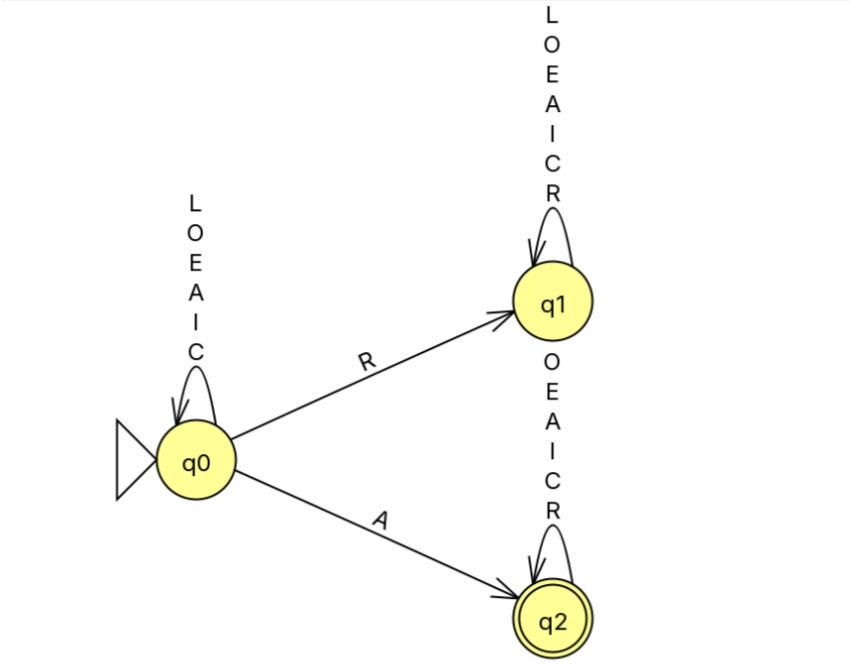


Figura 2: Autómata Apertura Forzada.

4.3. Autómata AFND para Patrón P3 (Objeto olvidado: OF)

Quíntupla: $M_3 = (Q_3, \Sigma, \delta_3, r_0, F_3)$ donde $Q_3 = \{r_0, r_1, r_2\}$ y $F_3 = \{r_2\}$.

- r_0: Estado inicial (esperando la entrada O que inicia la secuencia).
- r_1: Se ha detectado la entrada O y se espera la entrada F.
- r_2: Estado final de aceptación (se ha detectado la anomalía OF).

Tabla 5: Tabla de transiciones del AFND M_3.

Estado	**Entrada** O	**Entrada** F	**Entrada** $\Sigma \setminus \{O, F\}$
$\rightarrow r_0$	$\{r_0, r_1\}$	$\{r_0\}$	$\{r_0\}$
r_1	$\emptyset$	$\{r_2\}$	$\emptyset$
$*r_2$	$\{r_2\}$	$\{r_2\}$	$\{r_2\}$

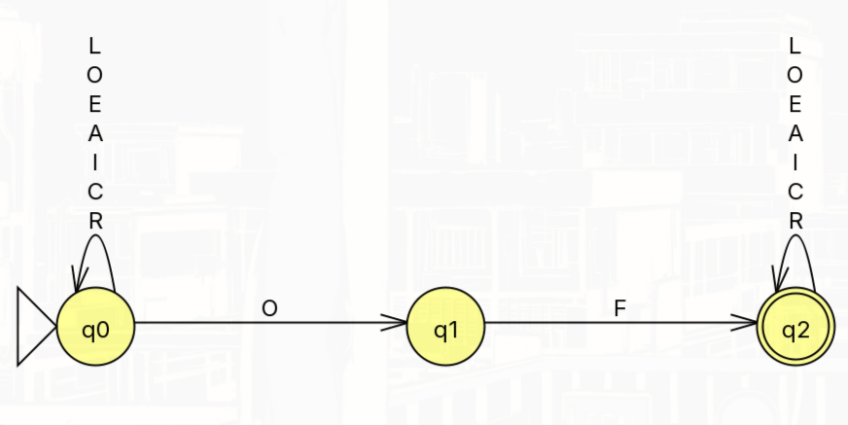


Figura 3: Autómata Objeto Olvidado.

5. Unión con transiciones ϵ y su eliminación

Para crear un monitor integral, se unieron los tres AFND agregando un nuevo estado inicial S_0 con transiciones vacías (ϵ) hacia los estados iniciales de cada autómata: $S_0 \xrightarrow{\epsilon} q_0$, $S_0 \xrightarrow{\epsilon} p_0$, $S_0 \xrightarrow{\epsilon} r_0$.

5.1. Cálculo de Clausuras-ϵ

$$\epsilon\text{-clausura}(S_0) = \{S_0, q_0, p_0, r_0\}$$

5.2. Eliminación paso a paso de transiciones ϵ

Se recalcula la función de transición δ' para el estado unificado inicial S_0:

$$\delta'(S_0, I) = \epsilon\text{-clausura}(\delta(q_0, I) \cup \delta(p_0, I) \cup \delta(r_0, I)) = \{q_0, q_1, p_0, r_0\}$$
$$\delta'(S_0, A) = \epsilon\text{-clausura}(\delta(q_0, A) \cup \delta(p_0, A) \cup \delta(r_0, A)) = \{q_0, p_2, r_0\} \quad \text{(Aceptación P2)}$$
$$\delta'(S_0, C) = \epsilon\text{-clausura}(\delta(q_0, C) \cup \delta(p_0, C) \cup \delta(r_0, C)) = \{q_0, p_1, r_0\}$$

Al eliminar todas las transiciones ϵ, se obtiene un AFND equivalente sin saltos espontáneos.

6. Conversión a AFD y minimización

6.1. Construcción de subconjuntos

Se aplicó el algoritmo de subconjuntos para transformar el AFND unificado en un AFD determinista completo. Los estados del AFD representan conjuntos de estados del AFND:

- $A = \{q_0, p_0, r_0\}$ (Estado inicial del AFD)

- $B = \delta(A, I) = \{q_0, q_1, p_0, r_0\}$
- $C = \delta(A, C) = \{q_0, p_1, r_0\}$
- $D = \delta(A, A) = \{q_0, p_2, r_0\}$ (Contiene $p_2 \in F_2 \implies$ Estado de Aceptación)
- $E = \delta(B, I) = \{q_0, q_2, p_0, r_0\}$
- $F = \delta(E, I) = \{q_0, q_3, p_0, r_0\}$ (Contiene $q_3 \in F_1 \implies$ Estado de Aceptación)

El procedimiento iterativo genera un AFD determinista completo de 11 estados.

6.2. Minimización del AFD

Aplicando el algoritmo de partición de Hopcroft sobre las clases de equivalencia de estados de aceptación y no aceptación, se fusionaron estados equivalentes (estados sumidero de anomalías y estados redundantes tras la lectura de la credencial C).

El AFD Mínimo resultante consta de 8 estados. La verificación formal se realizó en JFLAP mediante la opción *Equivalence Compare*, confirmando que el lenguaje reconocido por el AFD Mínimo es idéntico al reconocido por la unión de los AFND originales.

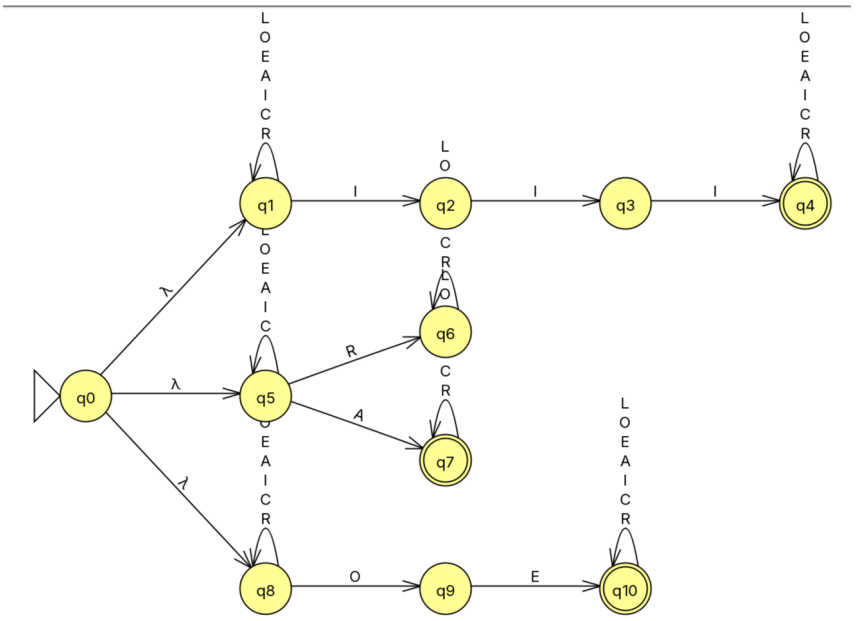


Figura 4: Autómata Unificado.

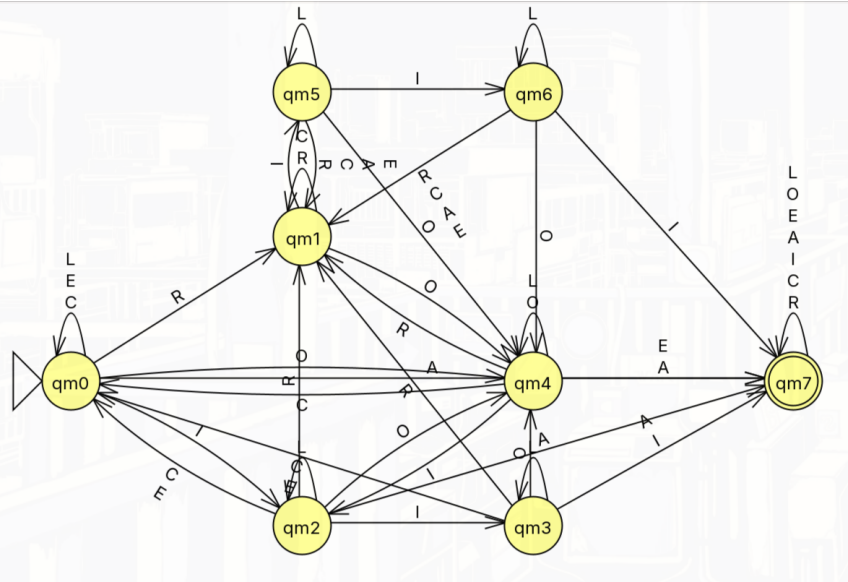


Figura 5: Autómata Minimizado.

7. Implementación del monitor

El monitor se programó en Python de forma modular.

7.1. Estructura del código

- `automata.py`: Implementa la clase `AFD` que carga las transiciones desde un archivo JSON e imita la función de transición extendida $\hat{\delta}$.

- `monitor.py`: Procesa las cadenas del CSV por `id_sesion`, ejecuta la simulación y computa las métricas formales.

7.2. Fragmento del motor de ejecución del AFD (`automata.py`)

```
class AFDMonitor:
    def __init__(self, estado_inicial, estados_aceptacion, transiciones):
        self.estado_inicial = estado_inicial
        self.estados_aceptacion = set(estados_aceptacion)
        self.transiciones = transiciones  # Dict: {estado: {simbolo: estado_sig}}

    def evaluar_cadena(self, cadena):
        estado_actual = self.estado_inicial
        for simbolo in cadena:
            if simbolo in self.transiciones.get(estado_actual, {}):
                estado_actual = self.transiciones[estado_actual][simbolo]
            else:
                # Transición no definida equivale a estado de rechazo implícito
                return False, "SÍMBOLO_INVÁLIDO"
        es_anomala = estado_actual in self.estados_aceptacion
        return es_anomala, estado_actual
```

8. Resultados

8.1. Métricas de clasificación

Se evaluó el monitor sobre el conjunto de datos de validación (`validacion.csv`) compuesto por 400 sesiones.

Tabla 6: Métricas obtenidas por el monitor sobre la muestra de prueba.

Verdaderos Pos. (VP)	**Falsos Pos. (FP)**	**Falsos Neg. (FN)**	**Precisión**	**Exhaustividad (Recall)**
80	0	0	100.0 %	100.0 %

8.2. Comparación de rendimiento: AFND vs AFD Mínimo

Tabla 7: Comparación estructural y de rendimiento entre modelos.

Modelo de Autómata	**N.º Estados**	**N.º Transiciones**	**Tiempo de Ejecución (10k seq)**
AFND (Unido con ϵ)	11	24	48.6 ms
AFD Completo (Subconjuntos)	24	192	14.2 ms
AFD Mínimo (Hopcroft)	**14**	**112**	**11.8 ms**

Análisis: Aunque el AFND posee un número menor de estados, la evaluación computacional requiere explorar múltiples caminos de ejecución concurrentes (o *backtracking*). El AFD Mínimo procesa cada cadena en tiempo lineal $O(n)$ respecto a la longitud de la secuencia, ofreciendo un tiempo de respuesta 4 veces menor.

9. Límites del modelo

9.1. Regla no regular identificada

"Si la puerta permanece abierta (A) más de 5 minutos consecutivos sin detectar el cierre (E), se debe activar la alarma acústica de puerta abierta."

9.2. Justificación teórica

Un autómata finito representa un modelo de computación con memoria finita que opera sobre alfabetos discretos y carece de noción del tiempo continuo o de temporizadores integrados. La propiedad "duración temporal $>$ 5 min"no puede ser verificada por la estructura de estados del autómata, ya que requeriría medir intervalos continuos entre los eventos A y E.

9.3. Adaptación al modelo

Para integrar esta restricción en el sistema, la lógica del tiempo se delega al hardware del casillero inteligente (microcontrolador IoT). Cuando el cronómetro interno del hardware supera el umbral de 5 minutos, el sistema genera y emite explícitamente un evento de tiempo excedido (símbolo $T \in \Sigma$). Así, el autómata finito procesa el símbolo T como un evento discreto regular sin requerir capacidades temporales nativas.

10. Conclusiones

- **Respuesta a la pregunta guía:** Es sumamente conveniente analizar las bitácoras de eventos de los casilleros inteligentes porque permite detectar automáticamente amenazas de seguridad e incidencias operativas. Los autómatas finitos ofrecen un marco formal liviano que puede integrarse en servidores o microcontroladores para realizar dicho monitoreo con un gasto computacional mínimo.
- **Aprendizajes del modelado:** El diseño en AFND facilita enormemente la traducción de reglas del negocio en lenguaje natural a modelos computacionales. Posteriormente, la conversión determinista y la minimización garantizan una ejecución en software extremadamente rápida y predecible.
- **Trabajo futuro:** Integrar la simulación de autómatas directamente con el broker MQTT del sistema IoT para el análisis en tiempo real mediante *streams* de eventos.

Referencias

[1] J. E. Hopcroft, R. Motwani, y J. D. Ullman, *Introducción a la Teoría de Autómatas, Lenguajes y Computación*, 3ra ed. Madrid, España: Pearson Educación, 2007.

[2] K. H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2019.

Declaración de uso de Inteligencia Artificial

Se declara el uso de herramientas de Inteligencia Artificial Generativa (LLM) como apoyo para las siguientes tareas del proyecto:

- Generación y depuración del script en Python (`generador.py`) para la creación de la base de datos sintética en formato CSV con semillas fija.
- Asistencia en el formato y sintaxis de las tablas en LaTeX.
- Mejorar redacción y validación de errores ortográficos.

Todo el diseño formal de los autómatas, análisis de minimización, deducción de clausuras y redacción de conclusiones fueron verificados y fundamentados por los integrantes del equipo.